



# Nodehaus

Building a Multilingual Agent Evaluation Framework

# About Me

**Peter Bouda**

+20 years of AI products & research

NLP product developer

Previously Head of AI Lab @ Apiax

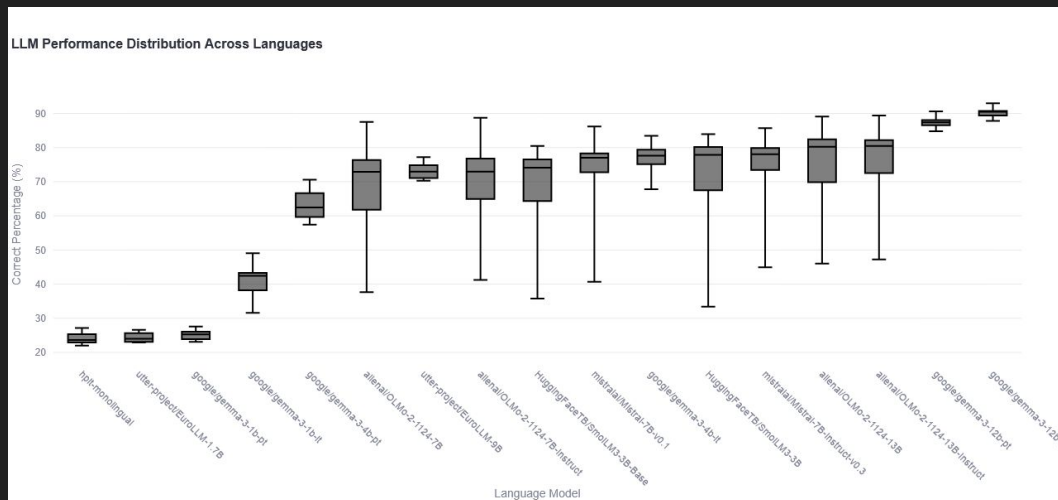
Co-founder of Nodehaus



# Context

## The Great European AI Language Championship ([substack](#))

- Testing smaller LLMs on multiple choice question answering
- 8 European languages
- Winner: Google's Gemma 3 12b



# **We're interested in EU technology sovereignty & language diversity**

The EU is pushing for homegrown tech alternatives, and we want to contribute

# Multilingual Evaluation Benchmarks

EuroEval: European language model benchmarking framework ([euroeval.com](https://euroeval.com))

- 13 Monolingual + 4 Multilingual evaluation datasets
- No agentic evaluation tasks

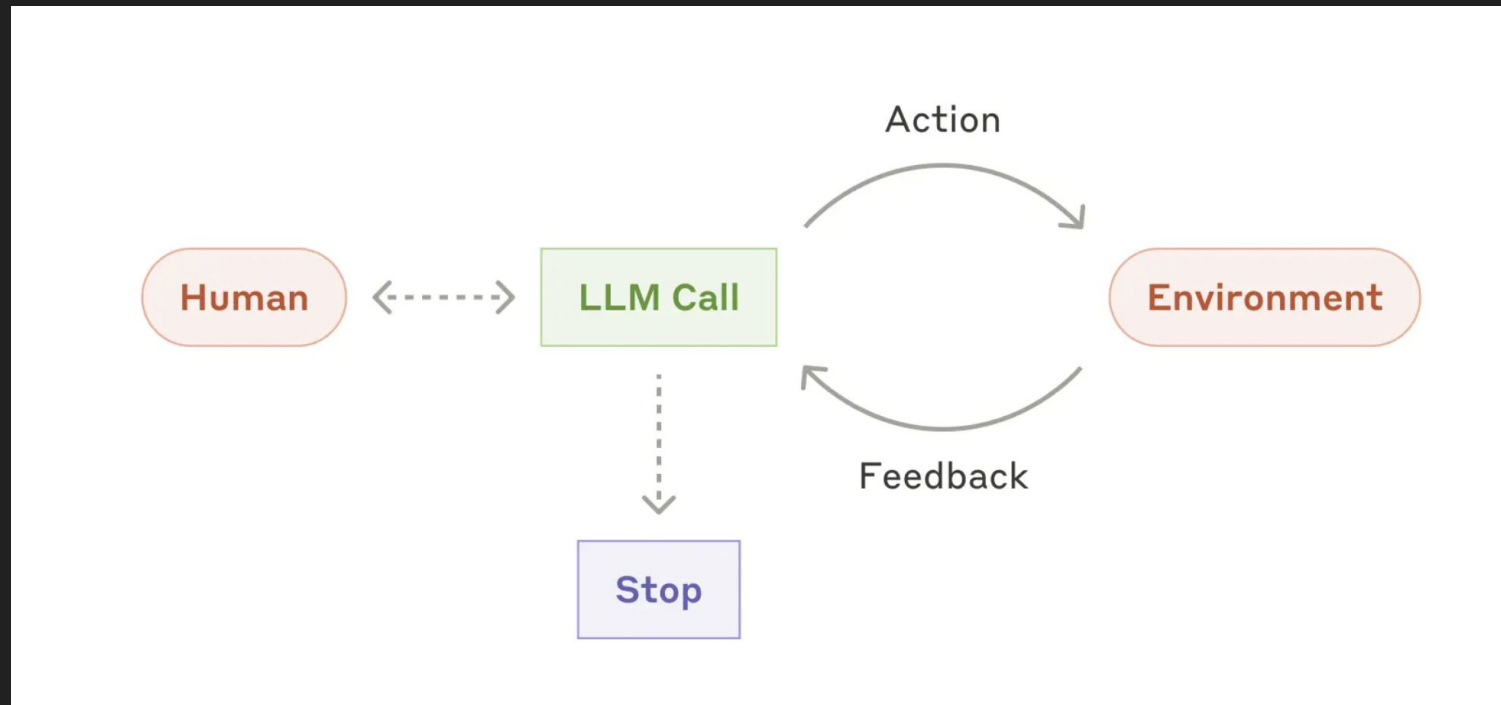
MAPS: A Multilingual Benchmark for Global Agent Performance and Security ([arxiv](https://arxiv.org/abs/2408.12811))

- German, Spanish, Portuguese, Japanese, Russian, Italian, Arabic, Hebrew, Korean, and Hindi
- Dataset not yet published

**There was no way to evaluate LLM  
agents in multilingual environments.**

So we (quickly) built our own.

# The Esoteric AI agent



# Anatomy of a simple tool calling agent

Ask question about the weather in a specific locations

- *What's the weather in Porto tomorrow?*

The agent has access to two tools:

- `get_current_date()` - returns the current date (YYYY-MM-DD)
- `weather_forecast(city: str, date: str)` - returns relevant weather data from an API

To successfully answer the question, the agent needs to call the tools in the correct order:

- Calculate dates
- Place location names in English

[See code in github](#)



# Chat templates and tool calling

Chat templates describe the format and syntax that was used when training the LLM

Dictates how **user**, **assistant**, **tools**, and **actions** are represented

Ollama and more high-level tools handle templating for you

We used Transformers - more control and visibility over the chat templates

# Chat templates and tool calling - Qwen3-8B

1.

```
<|im_start|>system
# Tools
```

You may call one or more functions to assist with the user query.

You are provided with **function** signatures within `<tools></tools>` XML tags:

```
<tools>
{ "type": "function", "function":
  { "name": "get_current_date",
    "description": "Get the current date.",
    "parameters": { "type": "object",
      "properties": {}, "return": { "type":
        "string", "description": "Current date
        in YYYY-MM-DD format" } } } }
</tools>
```

For each **function** call, return a json object with **function** name and arguments within `<tool_call></tool_call>` XML tags:

```
<tool_call>
{ "name": <function-name>, "arguments":
  <args-json-object> }
</tool_call><|im_end|>
```

2.

```
<|im_start|>user
How will the weather be in Munich
tomorrow?<|im_end|>
<|im_start|>assistant
<tool_call>
{ "name": "get_current_date",
  "arguments": {} }
</tool_call><|im_end|>
```

3.

```
{
  "role": "assistant",
  "tool_calls": [
    {
      "type": "function",
      "function": {
        "name": "get_current_date",
        "arguments": {}
      },
      "id": "I68auS2iR"
    }
  ],
  {
    "role": "tool",
    "content": "2025-09-11",
    "tool_call_id": "I68auS2iR"
  }
}
```

1. System prompt

2. User message + Assistant answer

3. Tool call and result are added to the conversation history

4. The tool response

4.

```
<|im_start|>user
<tool_response>
2025-09-11
</tool_response><|im_end|>
```

# Chat templates and tool calling - GPT-OSS-20B

1.

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-09-10
```

Reasoning: medium

*# Valid channels: analysis, commentary, final. Channel must be included for every message.*

Calls to these tools must go to the commentary channel:

```
'functions'.<|end|><|start|>developer<|message|># Tools
```

## functions

```
namespace functions {
```

```
// Get the current date.  
type get_current_date = () => any;
```

```
} // namespace functions<|end|>
```

2.

```
<|start|>user<|message|>How will the weather be in Munich tomorrow?<|end|>  
<|start|>assistant  
to=functions.get_current_date<|channel|>commentary  
json<|message|>{<|call|>
```

3.

```
{  
  "role": "assistant",  
  "tool_calls": [  
    {  
      "type": "function",  
      "function": {  
        "name": "get_current_date",  
        "arguments": {}  
      },  
      "id": "I68auS2iR"  
    }  
  ],  
  {  
    "role": "tool",  
    "content": "2025-09-11",  
    "tool_call_id": "I68auS2iR"  
  }  
}
```

1. System prompt

2. User message + Assistant answer

3. Tool call and result are added to the conversation history

4. The tool response

4.

```
<|start|>functions.get_current_date  
to=assistant<|channel|>commentary<|message|>"2025-09-11"<|end|><|start|>  
assistant
```

# First attempt at evaluating with DeepEval

Open-source LLM evaluation framework ([deepeval.com](https://deepeval.com)) - Synthesizer to create evaluation data

Supports conversation but not tool calls

Agentic Eval Metrics: Task Completion, Tool Correctness, Argument Correctness

Uses *LLM as judge* for Task Completion and Argument Correctness (default OpenAI GPT-4.1)

We decided to implement our own using Claude Code

***You're absolutely right!***



# Our evaluation framework: Custom Data + Metrics

Claude code workflow:

- Prompting to create English data
- Human review
- Translate language by language

Data Categories: No tool calls (10), absolute date (15), relative date (20)

Metrics: **Tool Correctness** and **Argument Correctness** (direct matches)

[View evaluation data in Github](#)

# How our evaluation and metrics works

We test whether the agent uses the right tools, with the correct arguments, or responds that it can't help when a request falls outside its scope.

| Prompt                                               | Tool: current_date | Tool: get_weather_forecast          | Response                                          | Pass |
|------------------------------------------------------|--------------------|-------------------------------------|---------------------------------------------------|------|
| What's the weather in New York on 07/15/2024?        |                    | city: New York<br>date: 2024-07-15  | temperature: 25<br>humidity: 75<br>wind_speed: 15 | Pass |
| What's the weather like in São Paulo tomorrow?       | 2025-09-11         | city: São Paulo<br>date: 2025-09-12 | temperature: 24<br>humidity: 85<br>wind_speed: 12 | Pass |
| How do I cook pasta?                                 |                    |                                     | I'm a weather bot [...]                           | Pass |
| How's the weather in London on September 15th, 2025? | 2025-09-11         | city: London<br>date: 2025-09-15    | temperature: 17<br>humidity: 70<br>wind_speed: 18 | Fail |
| Tell me a joke about Paris                           | 2025-09-11         | city: Paris<br>date: 2025-09-11     | temperature: 20<br>humidity: 65<br>wind_speed: 14 | Fail |

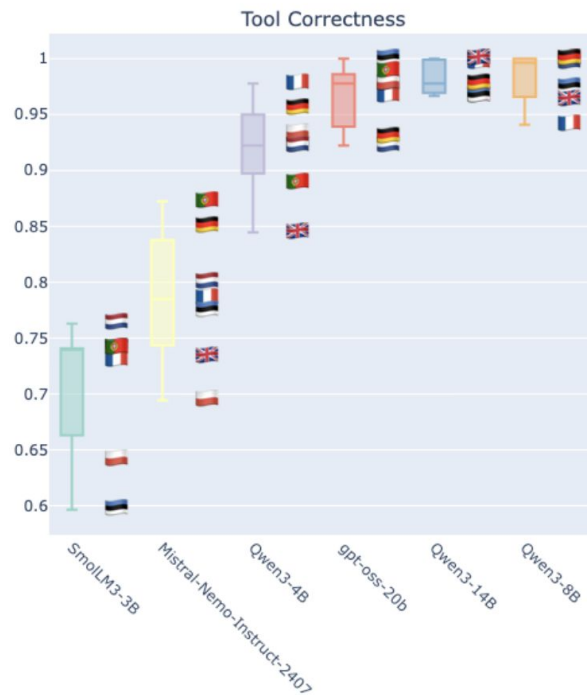
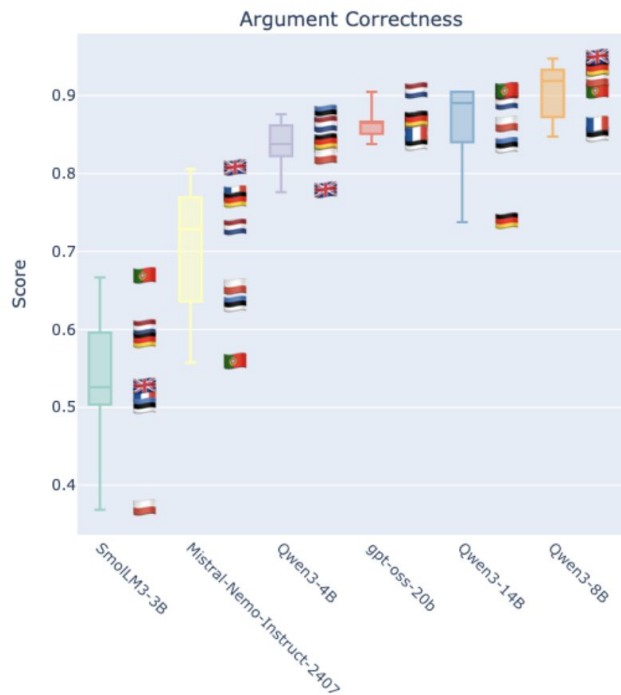
# Results

[Link to notebook](#)



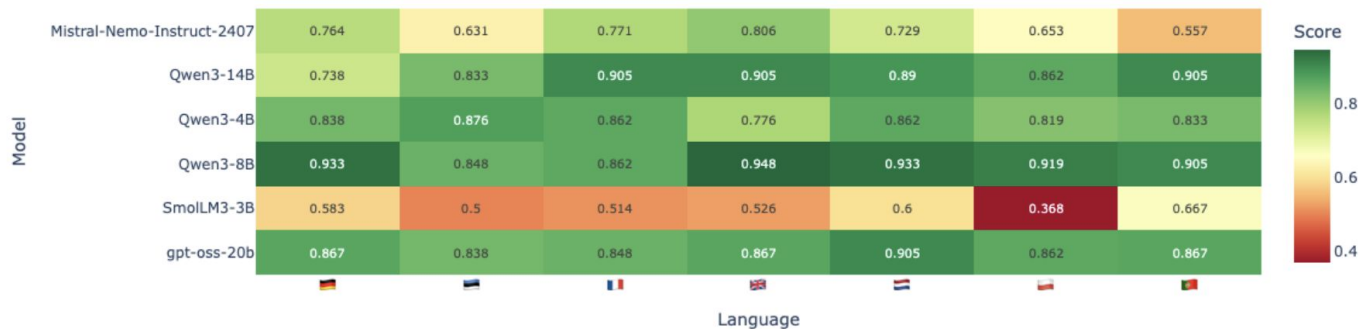
## English is not always the best performing language

Multilingual Agent Evaluation Results - All Metrics

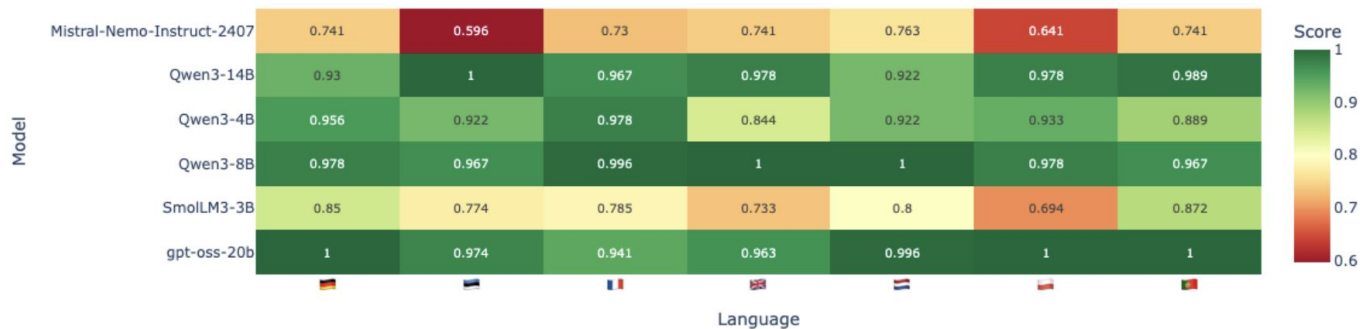


## Qwen3 8B on par with Qwen3 14B and GPT-OSS-20B

Argument Correctness Scores: Models vs Languages



Tool Correctness Scores: Models vs Languages

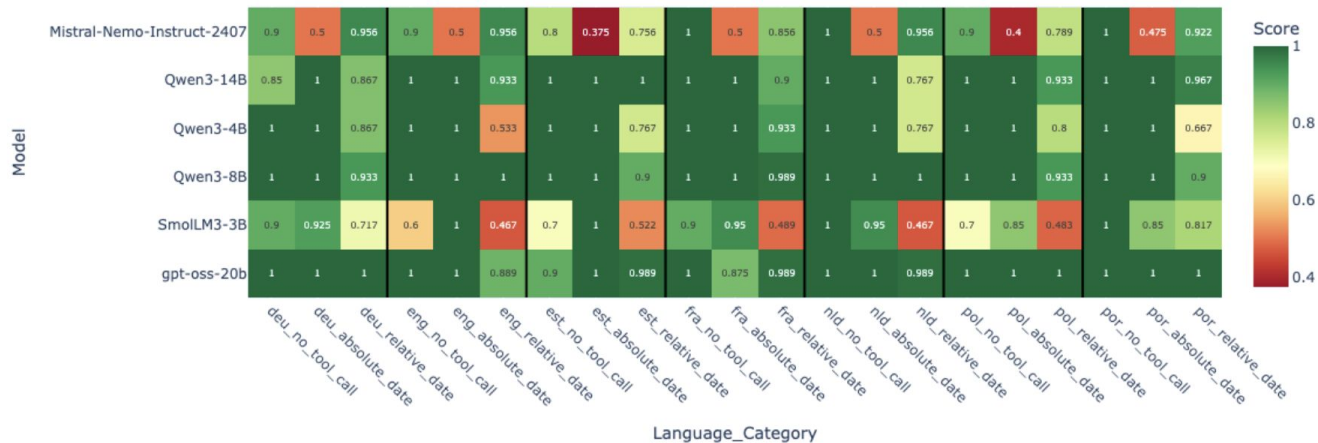


[Link to notebook](#)

Questions containing  
relative dates cause  
the most issues

Tool Correctness Scores: Models vs Languages/Categories

Categories: no\_tool\_call | absolute\_date | relative\_date



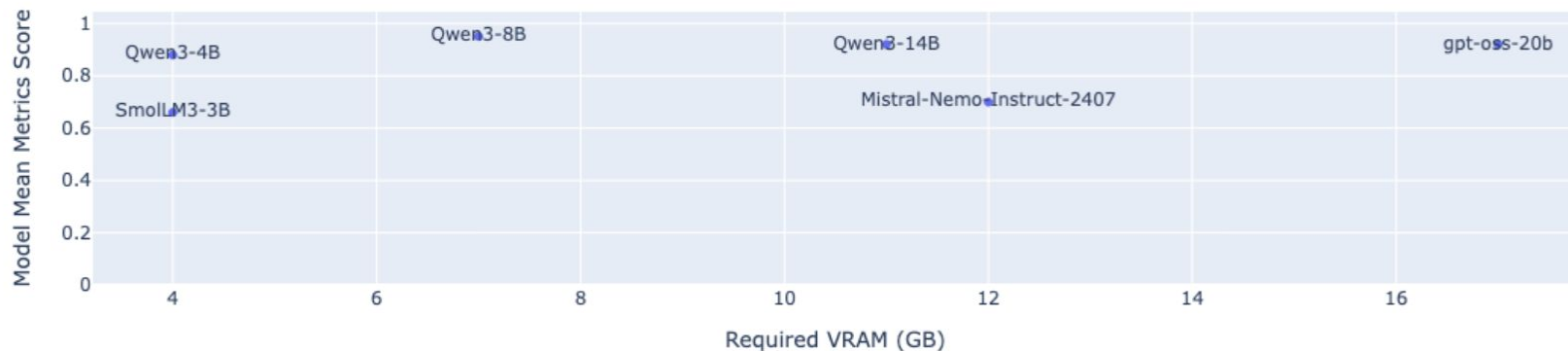
Argument Correctness Scores: Models vs Languages/Categories

Categories: no\_tool\_call | absolute\_date | relative\_date



Among small LLMs for multilingual agent tasks, Qwen3 8B offers the best performance-to-VRAM ratio

Model Performance vs Required VRAM



# Summary & Future Work

- Agentic performance is good across tested languages - lesser-resourced languages have a slight disadvantage
- Building your own custom evaluation takes less than a week because AI can help with the dataset: essential to know your models to get the best for your use case!
- Add inference speed to evaluation in the future
- We will be working on fine-tuning models for language + task combinations

# **Thank you!**

**Contact:**

[hello@nodehaus.io](mailto:hello@nodehaus.io)